Facility Maintenance Platform — Technical Documentation

Audience: engineers who will maintain, extend, or debug this codebase. **Scope:** the full stack — `client/` (React SPA) and `server/` (Express API) — as a single monorepo.

---

1. Overview

A facility maintenance company's public website plus a client/admin portal. Three user tiers:

- **Public visitors** — browse services, parts, FAQs, blog, testimonials; submit a contact form.
- **Clients** (authenticated, `role = 'client'`) — view service history/equipment, submit service requests, buy parts/plans online, track and cancel orders, leave feedback.
- **Admins** (`role = 'admin'`) — manage almost all site content, respond to service requests with quotes, manage orders/fulfillment, review analytics, and moderate feedback — all from one dashboard, no code deploy required for content changes.

Tech stack

Layer	Technology
Frontend	React 18, Vite, Tailwind CSS, React Router v6, Framer Motion, Recharts, Stripe.js/React Stripe
Backend	Node.js, Express 4, MySQL (`mysql2`), JWT (`jsonwebtoken`), Stripe SDK, Nodemailer
Validation/Security	`express-validator`, `helmet`, `express-rate-limit`, `bcryptjs`, `multer`
Logging	`pino` / `pino-http`
Deployment	Docker Compose (Nginx-served client + Node server + MySQL), or manual (static build + Node process)

---

2. Repository layout

This is a monorepo. `client/` and `server/` are independently deployable but developed together.

```
facility-maintenance/
├── client/                  # Vite + React SPA
│   ├── src/
│   │   ├── components/
│   │   │   ├── Auth/         # Login, Register, ForgotPassword, ResetPassword
│   │   │   ├── Dashboard/    # Client + Admin dashboard panels (see §9)
│   │   │   ├── Layout/       # Navbar, Footer
│   │   │   ├── Blog/         # Public blog section (Home page)
│   │   │   ├── Testimonial/  # Public testimonials section (Home page)
│   │   │   └── UI/           # Shared primitives: Button, Spinner, EmptyState,
ErrorBoundary, StripeCheckoutForm, Dropdown
│   │   ├── pages/            # Route-level components (one per URL)
│   │   ├── context/          # AuthContext, ProtectedRoute
│   │   ├── hooks/             # useSiteSettings
│   │   ├── services/api.js   # THE single axios client — every HTTP call goes
through here
│   │   ├── utils/media.js    # Resolves uploaded-image relative paths against the
API origin
│   │   └── App.jsx            # Route table (lazy-loaded pages)
│   ├── Dockerfile             # Multi-stage build → static files served by Nginx
│   ├── nginx.conf             # SPA fallback routing + gzip + cache headers
│   └── .env.example
├── server/                  # Express API
│   ├── routes/                # One file per resource area (see §6)
│   ├── middleware/             # auth, adminOnly, asyncHandler, errorHandler,
rateLimiters, upload, handleValidation
│   ├── config/                # env, db (MySQL pool), logger, mailer, stripe
│   ├── utils/                 # resourceRouter (generic CMS CRUD factory), orders
(cancel/refund), paginate, csv
│   ├── db/
│   │   ├── schema.sql         # Full schema + seed data — source of truth for a
FRESH database
│   │   └── migrations/        # Numbered, one-way migrations for an EXISTING
database (see §5.3)
│   ├── scripts/migrate.js     # Applies schema.sql
│   ├── uploads/               # User-uploaded images (gitignored; Docker-volume-
mounted in production)
│   ├── Dockerfile
│   └── .env.example
├── docker-compose.yml         # mysql + server + client, wired together
├── .env.example                # Docker Compose variable substitution (separate
from client/server .env files)
└── README.md
```

3. Local development setup

Without Docker

```
# 1. Database
mysql -u root -p -e "CREATE DATABASE facilitypro"
cd server && npm run db:migrate      # applies db/schema.sql

# 2. Server
cp .env.example .env                  # fill in real values (see §4)
npm install
npm run dev                           # nodemon, http://localhost:4000

# 3. Client (separate terminal)
cd ../client
cp .env.example .env
npm install
npm run dev                           # vite, http://localhost:5173
```

With Docker

```
cp .env.example .env    # root-level, for docker-compose variable substitution
docker compose up -d --build
```

Builds both Dockerfiles, starts MySQL with `server/db/schema.sql` auto-applied via `docker-entrypoint-initdb.d` on first boot, and persists the database + `server/uploads/` in named volumes. Client on `:8080`, API on `:4000`. **`VITE_API_URL` and `VITE_STRIPE_PUBLISHABLE_KEY` are baked into the client at *build* time** (Vite convention) — if you change them, rebuild (`docker compose up -d --build client`), a restart alone won't pick up new values.

---

4. Environment variables

There are **three separate `.env` files**, not one — don't confuse them:

File	Used by	Notes
`server/.env`	`npm run dev` in `server/`	Real secrets: DB creds, `JWT_SECRET`, Stripe keys, SMTP creds
`client/.env`	`npm run dev` in `client/`	`VITE_*` vars only, baked in at build time
`.env` (repo root)	`docker compose up`	Superset of both, read via `${VAR}` substitution in `docker-compose.yml`

Key server variables (see `server/config/env.js` — **the process refuses to boot if any required var is missing**, and `JWT_SECRET` must be ≥32 chars):

Variable	Required	Purpose
`DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `DB_PORT`	✅	MySQL connection
`JWT_SECRET`	✅ (≥32 chars)	Signs auth tokens
`JWT_EXPIRES_IN`	— (default `1h`)	Consider `8h` + for an admin-heavy workflow — a 1h expiry mid-task is disruptive
`STRIPE_SECRET_KEY`	✅	Server-side Stripe calls
`STRIPE_WEBHOOK_SECRET`	—	**Required in production** for payment status to update from `pending` automatically (see §7.3)
`NODEMAILER_EMAIL`, `NODEMAILER_PASS`	—	If unset, emails are logged instead of sent (safe no-op for local dev)
`CLIENT_URL`	✅	Used in password-reset email links
`CORS_ORIGINS`	— (falls back to `CLIENT_URL`)	Comma-separated allow-list

---

5. Database

5.1 Schema source of truth

`server/db/schema.sql` — every `CREATE TABLE IF NOT EXISTS`, safe to run against a fresh database. It also seeds initial content (services, team, FAQs, plans, parts, guides) so the site isn't empty on first boot — everything it seeds is subsequently editable from the admin dashboard.

5.2 Tables

Auth - `users` — `id, name, email, password (bcrypt), role ('client'|'admin'), banned, reset_token (SHA-256 hash), reset_token_expiry, created_at`

Client operational data - `service_history` — read-only to clients; admin-populated (no UI for this yet — see §10 TODO) - `equipment` — same; note column is `last_service` (snake_case), not `lastService` - `service_requests` — customer-submitted; `status` (`Pending|In Progress|Completed|Cancelled`, **Title Case** — inconsistent with `payments.status` which is lowercase, deliberately preserved to match a live database this schema was reverse-engineered from) plus `quote_amount_cents`, `quote_message`, `quoted_at` for the admin-quote workflow (§7.4) - `feedback` — dual-purpose table: `type = 'feedback'` (client testimonial/review submissions) vs `type = 'contact'` (public contact-form messages). Only `feedback`-type rows with `status='approved'` are shown publicly. `is_read` is the unified "needs admin attention" flag used by `GET /api/admin/alerts` (§9.1).

Payments / orders - `payments` — one row per Stripe PaymentIntent. **Two independent status fields, on purpose:** - `status` (`pending|succeeded|failed|refunded|canceled`) — has money moved? - `fulfillment_status` (`pending|accepted|processing|shipped|delivered|canceled`) — has it shipped? - `plan_id` / `part_id` / `service_request_id` — exactly one is set, identifying what was purchased (see §7.1) - `tracking_number`, `cancel_reason`, `canceled_at`, `refunded_at` — order lifecycle audit trail (§7.5)

Blog (pre-existing feature, admin-managed) - `blogs` — `content` is `LONGTEXT` (not `TEXT`) — the rich-text editor embeds images as base64 data URIs, which routinely exceed `TEXT`'s 64KB cap.

Admin-managed marketing content (the CMS — see §8) - `services`, `team_members`, `faqs`, `service_plans`, `parts`, `knowledge_guides`, `knowledge_videos` — all share the same shape: `sort_order`, `is_active`, `created_at`, `updated_at`, and (where applicable) `image_url`. - `site_settings` — key/value store (`address`, `email`, `phone`, `working_hours`, `emergency_phone`), not a generic-CRUD resource (see `server/routes/settings.js`).

5.3 Migrations

`server/db/migrations/*.sql` are **numbered, one-way, run-once** scripts for reconciling an *already-running* database with schema changes — `schema.sql`'s `CREATE TABLE IF NOT EXISTS` silently no-ops on a table that already exists, so a column added to an existing table needs an explicit migration. Apply manually:

```
mysql -u root -p your_db < server/db/migrations/00N_description.sql
```

Existing migrations, in order, and *why* each exists (useful context for the next one):

1. `001_add_payment_catalog_columns` — added `plan_id` / `part_id` / `quantity` to `payments` for the catalog-based checkout.
2. `002_add_service_request_quoting` — added the quote workflow columns to `service_requests` + `service_request_id` to `payments`.

3. `003_allow_null_user_id_for_guests` — `payments.user_id` and `feedback.user_id` were `NOT NULL` on the pre-existing database, which broke guest checkout and the anonymous contact form outright (`STRICT_TRANS_TABLES` rejects the insert). Made nullable.
4. `004_add_feedback_is_read` — backs the admin alerts queue.
5. `005_widen_blog_content` — `TEXT` → `LONGTEXT` (see blogs table note above).
6. `006_add_order_fulfillment` — the fulfillment-status/cancel/refund columns (§7.5).

When you add a new migration: update `schema.sql` too, so a fresh install matches an upgraded existing install exactly. Verify by running `schema.sql` against a throwaway database after editing.

---

6. API reference

Base path for everything is the server root (e.g. `http://localhost:4000`). Auth via `Authorization: Bearer <JWT>`. All list/mutation endpoints validate input with `express-validator`; failures return `400 { message, errors: [{field, message}] }`.

6.1 `routes/auth.js` — `/api/auth`

Rate-limited (`authLimiter`: 10 req/15min/IP) — brute-force protection.

Method	Path	Auth	Notes
POST	`/register`	—	`{name, email, password}`; password ≥8 chars
POST	`/login`	—	Returns `{user, token}`
POST	`/forgot-password`	—	Always responds the same way whether or not the email exists (no user enumeration). Reset token is a random 32-byte hex value, **stored as its SHA-256 hash** — the raw token only ever exists in the emailed link
POST	`/reset-password`	—	`{token, password}`

6.2 `routes/client.js` — `/api/client` (customer-facing)

Method	Path	Auth	Notes
GET	`/service-history`	client	Own rows only
GET	`/equipment`	client	Own rows only
GET	`/payments`	client	Own order history
PUT	`/payments/:id/cancel`	client	Only while `fulfillment_status` is `pending` / `accepted`; auto-refunds via Stripe if already paid (§7.5)
POST	`/feedback`	client	`{comment}` → `feedback` table, `type='feedback'`
GET	`/service-requests`	client	Own rows, incl. quote fields
POST	`/service-requests`	client	`{service_type, description}`
POST	`/contact`	— (public)	`{name, phone, email, company?, equipment_type?, message}` → `feedback` table, `type='contact'`

6.3 `routes/admin.js` — `/api/admin` (all routes require `auth` + `adminOnly`)

Method	Path	Notes
GET	`/users`	Paginated (`?page=&limit=` , max 100)
PUT	`/users/:id/role`	`{role: 'admin'|'client'}`
PUT	`/users/:id/ban`	`{banned: boolean}`
GET	`/payments`	Paginated, includes fulfillment fields + `user_email`
PUT	`/payments/:id/status`	`{fulfillment_status, tracking_number?, cancel_reason?}` ; `canceled` triggers refund if paid; emails customer on every change (§7.5)
GET	`/service-requests`	Paginated
PUT	`/service-requests/:id`	`{status?, quote_amount_cents?, quote_message?}` ; emails customer when a quote is set (§7.4)
GET	`/contact-messages`	Paginated, `type='contact'` only
PUT	`/contact-messages/:id/read`	Clears the alert
GET	`/feedback`	Paginated, `type='feedback'` only
POST	`/feedback/approve/:id` / `/feedback/reject/:id`	Sets `status` + `is_read=1`
GET	`/alerts`	Aggregates unread contact messages + unreviewed feedback + unquoted service requests (§9.1)
GET/POST/PUT/DELETE	`/blogs`	Standard CRUD; `content` accepts arbitrary-length HTML from the rich-text editor

6.4 `routes/reports.js` — `/api/admin/reports` (auth + adminOnly)

Method	Path	Notes
GET	`/summary?from=&to=`	Revenue-by-day, payments-by-status, service-requests-by-status, feedback-by-status, users-by-day, all scoped to the date range (default: trailing 30 days)
GET	`/export/:type?from=&to=`	CSV download; `type` ∈ `payments|service-requests|feedback|contact-messages|users`

6.5 `routes/public.js` — `/api/public` (no auth)

Method	Path	Notes
GET	`/feedback/approved`	3 random approved testimonials
GET	`/blogs`	Latest 6 posts

6.6 `routes/payments.js` — `/api/payments`

Method	Path	Auth	Notes
POST	`/create`	optional	See §7.1 — server always computes the amount
POST	`/webhook`	Stripe signature	Mounted **before** the global JSON body parser (needs the raw body) — see `server.js`

6.7 CMS resource endpoints — `server/routes/content.js`

For each of `services`, `team` (table `team_members`), `faqs`, `service-plans`, `parts`, `knowledge-guides`, `knowledge-videos`:

- `GET /api/public/<resource>` — active rows only, ordered by `sort_order`
- `GET /api/admin/<resource>` — all rows (auth + adminOnly)
- `POST /api/admin/<resource>` — create (multipart/form-data if the resource has an image)
- `PUT /api/admin/<resource>/:id` — partial update
- `DELETE /api/admin/<resource>/:id`

Plus `site_settings` via `routes/settings.js`: `GET /api/public/settings` (returns `{key: value}`), `GET /api/admin/settings`, `PUT /api/admin/settings` (bulk upsert, key allow-list enforced server-side).

7. Payments & order lifecycle

7.1 Payment integrity — the server always computes the amount

`POST /api/payments/create` accepts `{ kind: 'plan'|'part'|'service_request'|'custom', ... }`. **The client never sends an amount for `plan` / `part` / `service_request`** — the server looks up the referenced row's price itself:

- `kind: 'plan'` → `planId` → reads `service_plans.price_cents` (rejects with 400 if the plan has no price, i.e. "Contact Us"-only plans)
- `kind: 'part'` → `partId` + `quantity` → reads `parts.price_cents × quantity`
- `kind: 'service_request'` → `serviceRequestId`, **requires authentication**, verifies the request belongs to `req.user.id` and has been quoted → reads `quote_amount_cents`
- `kind: 'custom'` → the one deliberate exception: client-supplied amount, but bounded ($1–$50,000) and requires a description — for ad-hoc deposits that don't map to a catalog item

This is the load-bearing security property of the whole payments system: **never add a code path where the client dictates what gets charged.**

7.2 Stripe error handling — don't leak third-party status codes

`server/middleware/errorHandler.js` only trusts an error's status if the app explicitly set it (`err.appStatusCode`, via the `asError()` helper) — it does **not** trust a caught error's own `.statusCode` / `.status`. This mattered in practice: Stripe SDK errors carry Stripe's own HTTP status (e.g. `401` for an expired API key), and the old naive `err.statusCode || err.status || 500` fallback relayed that as *our* API's `401` — which the frontend's axios interceptor treats as "your session expired" and force-logs-out the user. `routes/payments.js` now explicitly catches Stripe failures and returns a generic `502` instead of letting the raw SDK error propagate. **If you add another third-party API call anywhere, wrap it the same way** — don't let its errors reach the global handler unfiltered.

7.3 Webhook — required for production

Without `STRIPE_WEBHOOK_SECRET` configured, `payments.status` never moves past `pending` on its own — nothing tells the server a card was actually charged. In production, configure a webhook endpoint (`<api>/api/payments/webhook`) in the Stripe dashboard pointing at this deployment, and set `STRIPE_WEBHOOK_SECRET`. Locally, use `stripe listen --forward-to localhost:4000/api/payments/webhook`. The handler maps `payment_intent.succeeded` / `.payment_failed` / `.canceled` → `payments.status`.

7.4 Service request quoting

`service_requests` starts `status='Pending'`, no quote. Admin sets `quote_amount_cents` / `quote_message` (customer is emailed), customer sees it on their dashboard and can pay via `kind: 'service_request'`. There's no "customer requests a re-quote" flow yet — see §10.

7.5 Order cancellation & refunds (`server/utils/orders.js`)

`cancelOrder(db, payment, reason)` is the single shared implementation, called from both the customer-cancel and admin-status-change routes:

1. If `payment.status === 'succeeded'`, calls `stripe.refunds.create()` **first**. If that throws, the function throws too — the caller returns `502` and the database is **not** touched. This ordering matters: a failed refund must never leave an order silently marked "canceled" while the customer is still charged.
2. Only after a successful refund (or if nothing was ever charged) does it update `status` / `fulfillment_status` / `cancel_reason` / `canceled_at` / `refunded_at`.

Customers can only cancel while `fulfillment_status` is `pending` or `accepted` (`CANCELABLE_FULFILLMENT_STATUSES` in `orders.js`) — once `processing` or later, they can't self-service cancel (matches standard e-commerce UX; there's no "return" flow post-shipment yet, see §10).

Revenue accuracy: `reports.js` 's revenue query sums only `status = 'succeeded'`. A refund moves status to `'refunded'`, so it's excluded from revenue automatically — no separate "negative line item" bookkeeping needed. `Reports.jsx` also surfaces the refunded total as its own stat tile so it's visibly netted out rather than silently vanishing.

---

8. The generic CMS resource pattern

`server/utils/resourceRouter.js` exports `createResourceRouter({ table, fields, hasImage, orderBy })`, which builds a full public-read + admin-CRUD router pair from a field-schema description. This is what all seven CMS resources (§6.7) are built from — see `server/routes/content.js` for the instantiation of each.

To add a new admin-manageable content type:

1. Add a `CREATE TABLE` to `schema.sql` (+ a migration if the DB is already running) with at minimum `id, sort_order, is_active, created_at, updated_at`.
2. In `content.js`, call `createResourceRouter({ table: 'your_table', fields: [...], hasImage: true|false })` and add it to the `RESOURCES` array.
3. On the client, add the field schema + a `ResourceManager` instance in `AdminDashboard.jsx` (`client/src/components/Dashboard/ResourceManager.jsx` is the generic admin CRUD UI — same idea as the backend factory, mirrored) and an `apiService` entry in `services/api.js` (`buildResourceApi(path, label, { hasImage })`).
4. Add a public-facing page/section that calls `apiService.get<Resource>()`.

No new route files, no new admin UI components — this is the intended extension point for "add another kind of manageable content."

---

9. Admin dashboard (client)

`client/src/components/Dashboard/AdminDashboard.jsx` — tabbed interface, default tab is **Alerts**.

9.1 Alerts tab (`AlertsPanel.jsx`)

Calls `GET /api/admin/alerts`, which aggregates unread contact messages + unreviewed feedback + unquoted service requests into one list, each with an inline action (mark read / approve / reject / respond-with-quote). Taking the action removes the item from the list. The tab badge count is reported to the parent via a `useEffect` keyed on the items array — **not** from inside the `setItems` updater function (an earlier version did this and triggered React's "Cannot update a component while rendering a different component" warning; don't reintroduce that pattern).

9.2 Reports tab (`Reports.jsx`)

Date-range-filtered charts (Recharts) fed by `/api/admin/reports/summary`, plus CSV export buttons that fetch a blob (`responseType: 'blob'` in `apiService.exportReport`) and trigger a download via a temporary object URL.

9.3 Content tab

One `ResourceManager` per CMS resource (§8) + `BlogManagement` (rich-text, separate because it doesn't fit the generic field-schema shape) + `SiteSettingsPanel` (key-value form, also doesn't fit the generic shape).

9.4 Users tab

`UserManagement.jsx` — role changes, ban/unban.

9.5 Operations tab

`PaymentsList` (order fulfillment control, §7.5), `ServiceRequestsList` (quote response, mirrors the Alerts panel's inline form), `ContactMessagesList` (read-only history), `FeedbackApproval` (read-only history — approval itself happens from Alerts once unread, or here if you want to review already-actioned items).

---

10. Known limitations / suggested next steps

- **No automated test suite.** Everything in this codebase has been verified manually (curl against a running server + real MySQL, confirmed via direct DB queries and, for payments, independently via Stripe's own API). Adding integration tests (`supertest` + a test database) around `routes/payments.js` and `utils/orders.js` would be the highest-value place to start, given the financial correctness requirements there.

- **`react-quill`'s pinned `quill` dependency has a known moderate XSS advisory** (`GHSA-4943-9vgg-gr5r`) with no upstream fix as of this writing. Accepted risk: the rich-text editor is admin-only, not public-facing input.
- **No refresh-token flow** — `JWT_EXPIRES_IN` is a flat expiry (default 1h in some deployments; consider raising it, see §4). A long admin session can expire mid-task.
- **`equipment` / `service_history` have no admin-facing create/edit UI** — currently only populated by direct DB access. If clients are expected to see their own equipment/service history, this needs an admin panel (would follow the same `ResourceManager` pattern, §8, scoped by `user_id`).
- **No "return after delivery" flow** — cancellation only works pre-shipment (§7.5). Post-delivery returns/refunds are a manual admin DB operation today.
- **Bundle size** — `Dashboard-*.js` (~690KB) and the shared `lucide-react` chunk (~555KB) are the two largest chunks post code-splitting. Public-facing pages are already small (5–20KB each, lazy-loaded); further gains would come from per-icon `lucide-react` imports instead of the barrel import, if it becomes a real problem.
- **Stripe webhook is not configured by default** — see §7.3. Without it, `payments.status` never leaves `pending`, which also means completed orders won't show as `succeeded` in Reports/revenue until the webhook is set up.

---

11. Coding conventions

- **Never trust client input for money.** See §7.1 — this is the one rule in this codebase with zero exceptions.
- **Wrap async route handlers in `asyncHandler`** (`server/middleware/asyncHandler.js`) — a bare `async (req, res) => {...}` that throws will crash the process on an unhandled rejection; `asyncHandler` forwards to Express's error middleware instead.
- **Don't let third-party errors set your HTTP status** — see §7.2.
- **Validate with `express-validator` + `handleValidation`** on every route accepting a body — the existing routes are the reference pattern.
- **New admin-manageable content → use the generic resource pattern** (§8) unless the shape genuinely doesn't fit (site_settings and blogs are the two exceptions, and both have a documented reason).
- **Migrations are additive and one-way.** Never edit a migration that's already been applied anywhere; write a new one.
- **Client `services/api.js` is the only place that calls `axios` directly.** Components call `apiService.xxx()`, never `axios` / `fetch` directly — this keeps the 401-triggers-logout interceptor and error-message normalization (`request()` helper) consistent everywhere.